DATAFLOW · SKEW BUFFERS · WEIGHT-STATIONARY FUNDAMENTALS

How data enters the array, waits in the delay buffers, and leaves

A cycle-by-cycle account of the 4×4 weight-stationary systolic multiplier — what the skew buffers are for, what they hold at every instant, what the DRAIN phase is actually draining, and where the 17 cycles go

Design `accelerator_system_top` · ROWS = COLS = 4, int8 × int8 → int32 · 100 MHz, single clock, no back-pressure
Job length **N + 13** cycles loading weights, **N + 9** reusing them (**17** / **13** for a 4×4 × 4×4 product)
Companion documents: `timing.pdf` (STA and performance), `architecture_and_dataflow.pdf` (module reference), `context_mkdwn/12_dataflow_and_timing.md` (constants)

Contents

WHO THIS IS FOR

Someone who can read the RTL but wants the *mental model*: why the data arrives diagonally, what is sitting in each delay line at any moment, and why the answer pops out aligned at the bottom. Every number here is taken from the RTL and confirmed by the QuestaSim regression (`verification_recheck/` , 11/11 testbenches, 3 680 checks, 0 mismatches).

1. The one-page answer

A systolic array is a grid of tiny processors that pass data to their immediate neighbours once per clock, like a heart pumping — that is where the name comes from. Nothing broadcasts, nothing has a long wire, and every value moves exactly one hop per cycle. That constraint is what makes it fast, and it is also what forces the awkward-looking skewed input.

In **this** design:

Stage	Cycles	What is happening
Weight load (WLOAD)	4	The 4×4 weight matrix is pushed down the array's vertical bus, one row per cycle, **bottom row first**. Each PE grabs the value passing through it into `weight_q`, where it then **stays for the rest of the job**.
Compute (COMPUTE)	N	One activation vector enters per cycle. The input skew buffer delays row *r* by *r* cycles so the vector enters the array as a **diagonal wavefront**, not a straight line.
Drain (DRAIN)	8	Nothing new enters. The last vector is still walking through the array, the de-skew buffer and the output write port. The FSM waits exactly as long as that pipeline is deep.
Done	1	The `done` pulse.

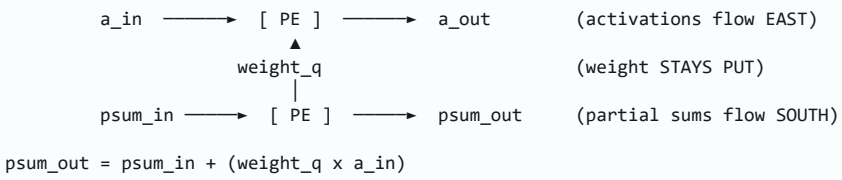
> **Total = ROWS + N + STORE_LATENCY + 1 = 4 + N + 8 + 1 = N + 13**
>
> For a 4×4 × 4×4 product (N = 4): **17 cycles = 170 ns at 100 MHz**
>
> Skip the weight load, because the array still holds the matrix from last time:
>
> **Total = N + 9 → 13 cycles = 130 ns**

The rest of this document explains each of those lines properly.

2. What "weight-stationary" means, and why anyone chose it

A matrix multiply `C = A × W` needs three things to meet inside each multiplier: an activation, a weight, and a running partial sum. Only one of them can afford to sit still. Whichever one you pin down names the dataflow.

Weight-stationary pins the weights. Each PE loads one weight once, and then that weight never moves again for the entire job. Activations flow past it horizontally; partial sums flow past it vertically; the weight just sits there being multiplied by whatever arrives.



Why that is the right choice here:

Property	Consequence
Weights are read once, used N times	The whole point. Load a 4×4 matrix in 4 cycles, then stream as many activation vectors past it as you like. The load cost amortises to nothing for large N — and in this design it can be **skipped entirely** for a repeat job (§14).
The weight never crosses a wire during compute	No fetch, no bandwidth, no energy. In the measured power report the entire 4×4 PE array burns **3 mW**; the memory holding the results burns 8 mW.
Partial sums accumulate in place as they fall	A column of 4 PEs computes a complete 4-term dot product with no adder tree and no accumulator register file — just one 32-bit add per PE.
Every link is register-to-register and one hop long	This is what lets the design close timing at 100 MHz with +0.714 ns of slack. There is no combinational path that spans the array.

3. The three dataflows compared

Dataflow	What stays put	Best when	Cost
Weight-stationary (this design, Google TPU v1)	The weight, in `weight_q`	One weight matrix is reused across many activation vectors — exactly a neural-network layer, or a filter applied to a whole image.	You pay a load phase whenever the weights change, and partial sums have to move.
Output-stationary	The accumulator for one output element	Very deep reductions, where you want the partial sum never to move.	Both weights and activations must stream in every cycle — double the input bandwidth.
Row/input-stationary (Eyeriss)	A row of the activation map	Convolution with heavy data reuse in two dimensions.	Much more complex control and interconnect.

The choice here is the natural one for a demonstrator: the weight matrix is small, fixed for a job, and the interesting dimension is "how many activation vectors can I push through".

4. Why a systolic array needs skewing at all

This is the part that confuses people first, so it is worth being slow about.

Consider computing `C[0][0] = A[0][0]·W[0][0] + A[0][1]·W[1][0] + A[0][2]·W[2][0] + A[0][3]·W[3][0]`. In the array, `W[r][0]` lives in the PE at row *r*, column 0. So the four products that make up `C[0][0]` are computed in **four different PEs, stacked vertically**, and their sum falls down the column.

Now the timing problem. The partial sum takes **one cycle per row** to fall. So:

- The product in **row 0** must be computed at some cycle *t*.
- Its partial sum arrives at row 1 at cycle *t*+1 — so row 1's product must be computed at *t*+1, not at *t*.
- Row 2 at *t*+2, row 3 at *t*+3.

But `A[0][0]`, `A[0][1]`, `A[0][2]` and `A[0][3]` are all elements of the **same** activation vector, which arrives from memory in **one** cycle as a single 32-bit word. If you fed all four into the array simultaneously, row 1 would multiply its weight by `A[0][1]` at cycle *t* — one cycle *before* the partial sum from row 0 arrives. The sum would be wrong.

> **THE CORE INSIGHT**
> The partial sum travelling down the column is a **MOVING DEADLINE**. Each row must present its activation at exactly the moment the partial sum from above reaches it. Since the sum moves one row per cycle, **ROW *R*'S ACTIVATION MUST BE DELAYED BY *R* CYCLES**. That delay is the entire job of the input skew buffer.

5. The input skew buffer — staggering the wavefront

`input_buf.sv` is four independent delay lines, one per array row:

```
for (r = 0; r < ROWS; r++) begin : gen_input_skew
    delay_pipe #(.WIDTH(WIDTH), .DELAY(r)) u_a_delay ( ... );   // activation
    delay_pipe #(.WIDTH(1),     .DELAY(r)) u_v_delay ( ... );   // its valid bit
end
```

Array row	DELAY	Storage	Note
0	0	**none — a plain wire**	`delay_pipe` with DELAY = 0 is a pass-through. Row 0 needs no delay because its partial sum starts at zero from the top edge.
1	1	1 register (8 bit + 1 valid)	
2	2	2 registers	
3	3	3 registers	The deepest line — the last row to receive its activation.

Total storage: **0 + 1 + 2 + 3 = 6 activation bytes and 6 valid bits**. That is the whole input buffer — tiny, because it holds only the triangle of data that is "waiting its turn".

What it holds, instant by instant

Feed vectors A0, A1, A2, A3 on consecutive cycles. Writing $A_k[r]$ for element *r* of vector *k*, this is what the array's west edge sees:

```
cycle      →   t      t+1     t+2     t+3     t+4     t+5     t+6
                ┌──────┬──────┬──────┬──────┬──────┬──────┬──────┐
row 0           │ A0[0]│ A1[0]│ A2[0]│ A3[0]│   -  │   -  │   -  │   delay 0
row 1           │   -  │ A0[1]│ A1[1]│ A2[1]│ A3[1]│   -  │   -  │   delay 1
row 2           │   -  │   -  │ A0[2]│ A1[2]│ A2[2]│ A3[2]│   -  │   delay 2
row 3           │   -  │   -  │   -  │ A0[3]│ A1[3]│ A2[3]│ A3[3]│   delay 3
                └──────┴──────┴──────┴──────┴──────┴──────┴──────┘
                   \      \      \      \
                    \      \      \      \      each diagonal is ONE activation vector
                     \      \      \      \     entering the array as a WAVEFRONT
                      A0     A1     A2     A3

Contents of the delay lines at cycle t+2:
    row 1 pipe : [ A1[1] ]                  1 byte  in flight
    row 2 pipe : [ A1[2], A0[2] ]           2 bytes in flight   <- A0[2] about to emerge
    row 3 pipe : [ A2[3], A1[3], A0[3] ]    3 bytes in flight
```

Read the table by diagonal, not by column. Vector A0 occupies cycles *t*…*t*+3 as it steps down the rows — that diagonal **is** the wavefront. The buffer's job is to convert a vertical column of data into a diagonal one.

> **THE VALID BIT TRAVELS WITH THE DATA**
>
> Each row delays `valid` by the same amount as the activation byte. This is not decoration: a PE accumulates only when `valid_in` is high, so if valid and data got separated by even one cycle the array would accumulate the wrong products. It is the reason `delay_pipe` is instantiated twice per row rather than the valid being computed centrally.

6. Inside the PE: two streams, one register file

Every one of the sixteen PEs is this (`MAC.sv`, condensed):

```
assign product_ext = 32'(weight_q * a_in);        // 8x8 signed multiply
assign adder_out   = psum_in + product_ext;       // 32-bit signed add

always_ff @(posedge clk) begin
    if (wload) weight_q <= psum_in[7:0];          // capture a weight, ONLY during load
    a_q     <= a_in;                              // activation marches east
    valid_q <= valid_in;                          // valid marches with it
    psum_q  <= valid_in ? adder_out : psum_in;    // accumulate, or just pass through
end

assign a_out    = a_q;
assign psum_out = wload ? 32'(weight_q) : psum_q; // vertical bus is time-multiplexed
```

Register	Written when	Purpose
`weight_q`	**only** while `wload`	The stationary weight. Survives COMPUTE, DRAIN, DONE and IDLE untouched — this is literally what makes the architecture "weight-stationary", and it is what allows the reuse path in §14.
`a_q`	every cycle	Passes the activation to the east neighbour one cycle later.
`valid_q`	every cycle	Passes the valid flag east, in step with `a_q`.
`psum_q`	every cycle	Holds this PE's contribution to the running dot product, on its way south.

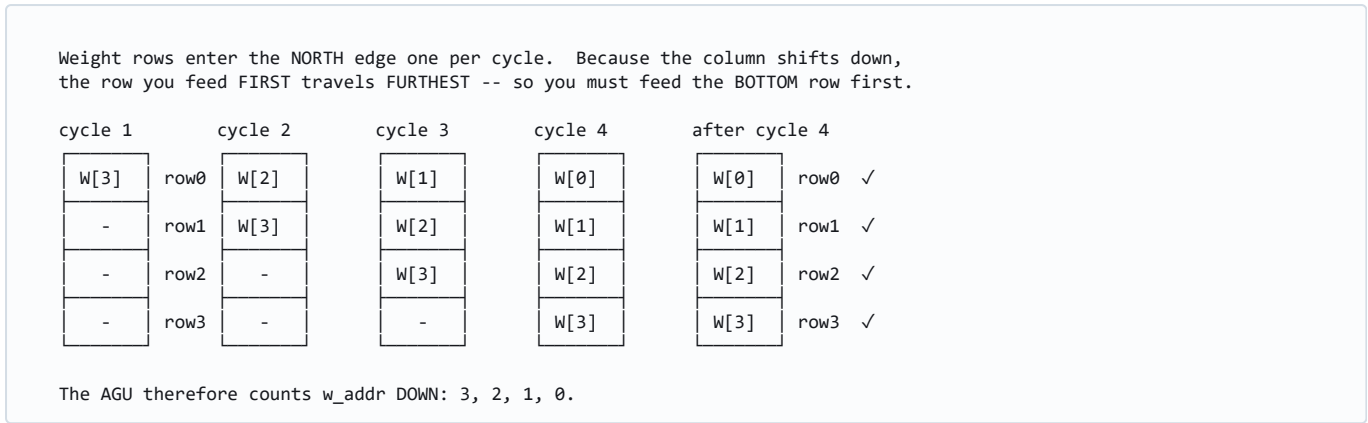
> **THE SINGLE MOST IMPORTANT LINE IN THE DESIGN**
>
> `psum_q <= valid_in ? adder_out : psum_in;` and `adder_out = psum_in + product_ext`.
>
> Notice what is *absent*: `psum_q` never appears on the right-hand side of the adder. The accumulation chain is fed **ENTIRELY FROM `PSUM_IN`**, i.e. from the PE above. A PE's own stored partial sum is only ever an output. That single fact is why stale data left in `psum_q` cannot corrupt a later result — see §13.

7. Loading the weights — the shift-register trick

There is no dedicated weight bus. During WLOAD the design **reuses the partial-sum bus** as a downward shift register, which is what the `wload` multiplexer on `psum_out` is for. While `wload` is high:

- `psum_out = weight_q` — each PE outputs the weight it is currently holding, instead of its partial sum;
- `weight_q <= psum_in[7:0]` — and simultaneously captures whatever is arriving from above.

Together those two lines make the whole column a shift register: one value enters the top each cycle, and everything already inside moves down one row.

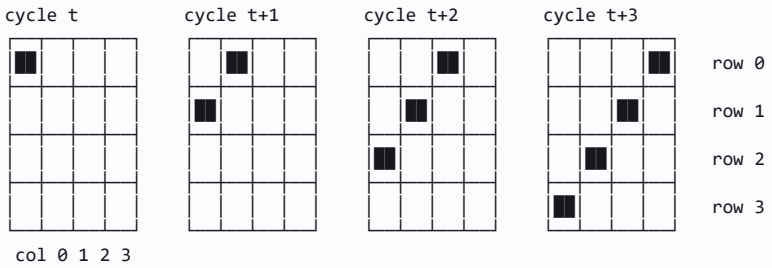


> **LOAD-BEARING RULE — BOTTOM ROW FIRST**
> Feed `W[ROWS-1]` first and `W[0]` last. Get this backwards and the design still runs, still reports `done`, and produces a plausible-looking but wrong matrix — it computes `A × flipud(W)`. A
> **SYMMETRIC**
> test matrix will not catch it, which is why the regression uses non-symmetric golden vectors.

Each PE captures on every one of the 4 cycles, so it holds several different weights in succession; only the last one it captures is the one it keeps. That is fine — and it is also the source of the "residue" discussed in §10 and §13, because those in-transit weight bytes are simultaneously being written into `psum_q`.

8. The compute wavefront, cycle by cycle

Once the weights are in place, each activation vector sweeps through the array diagonally. Track a single vector A0 entering at cycle *t*:




```
■ = a PE computing a product belonging to activation vector A0 this cycle.

The wavefront is an anti-diagonal moving SOUTH-EAST at one cell per cycle.
Reading it two ways:
  * down a COLUMN  : the four products of one output element C[.][c], summing as they fall
  * along a ROW    : one activation byte being reused against four different weights
```

Two independent things are being reused here, and that double reuse is the whole efficiency argument for a systolic array:

Reuse	Direction	What it saves
One weight, many activations	stationary	Weight fetch bandwidth — zero after the load phase.
One activation, many weights	eastward	Activation fetch bandwidth — read once from `a_mem`, used in 4 PEs.
Partial sums accumulate in place	southward	No adder tree, no accumulator file, no write-back until the answer is complete.

Because column *c* is reached *c* cycles after column 0, and the sum then has to fall ROWS−1 = 3 rows, the complete dot product for column *c* emerges from the bottom of the array at:

```
array latency for column c = (ROWS − 1) + c = 3 + c cycles after the vector entered
→ column 0 at +3, column 1 at +4, column 2 at +5, column 3 at +6
```

9. The output de-skew buffer — putting the staircase back together

The results leave the bottom of the array in a **staircase**: column 0 first, column 3 three cycles later. But the output memory is 128 bits wide and stores all four lanes of a result row in one word, so they must be presented together. `output_skewbuf.sv` is the mirror image of the input buffer:

Column	Emerges at	De-skew DELAY	Storage	Aligned at
0	+3	COLS−1−c = **3**	3 × 32-bit regs	+6
1	+4	**2**	2 × 32-bit regs	+6
2	+5	**1**	1 × 32-bit reg	+6
3	+6	**0** — a wire	none	+6

```
BEFORE de-skew (psum_raw)            AFTER de-skew (result[])
staircase out of the array           one aligned 4-lane vector

cyc  +3  +4  +5  +6                  cyc  +3  +4  +5  +6
c0 | C0 |    |    |                  c0 |    |    |    | C0 |   held 3 cycles
c1 |    | C1 |    |                  c1 |    |    |    | C1 |   held 2 cycles
c2 |    |    | C2 |                  c2 |    |    |    | C2 |   held 1 cycle
c3 |    |    |    | C3 |             c3 |    |    |    | C3 |   straight through
                  ↑                                     ↑
     each lane finishes at its own   all four lanes valid on the SAME cycle,
     time -- unusable as a word      ready to be packed into one 128-bit word
```

Total storage: **3 + 2 + 1 + 0 = 6 words of 32 bits** — the exact mirror of the input buffer's 6 bytes. The delays are complementary by construction: the array adds `+c`, the de-skew adds `+(COLS−1−c)`, and the sum is `COLS−1 = 3` for every column. Add the ROWS−1 = 3 for falling down the column and you get the constant that governs the whole design:

> **L1 result latency = (ROWS − 1) + (COLS − 1) = ROWS + COLS − 2 = 6 cycles,** independent of column
> + 1 for the `a_mem` read → **L2 result latency = 7**
> + 1 for the write-strobe alignment → **STORE_LATENCY = 8**

> **WHY DE-SKEW RATHER THAN FOUR SEPARATE WRITE PORTS**
> You could skip the de-skew buffer and write each lane as it arrives, using byte-enables on the output BRAM. It would *not* make the job shorter: the last column still emerges at +6, so DRAIN would be unchanged. What you would lose is the single clean "one result word per cycle" interface that makes the AGU's write-strobe pipe a plain shift register. Six 32-bit registers is a cheap price for that.

10. What DRAIN is actually draining

This is the question the phase name answers badly. DRAIN is **not** flushing garbage. It is the FSM sitting still and waiting, because **the last activation vector issued is still in flight**.

When the FSM leaves COMPUTE, the last vector has only just been *addressed*. It still has to:

#	Stage still to traverse	Cycles	Where it is buffered
1	`a_mem` synchronous read	1	The BRAM's own output register
2	Input skew delay for the deepest row	up to 3	`input_buf` delay lines
3	Falling down the PE column	3	`psum_q` in each PE
4	Travelling east to the last column	up to 3	`a_q` in each PE
5	De-skew alignment	0–3	`output_skewbuf` delay lines
6	Write-strobe alignment into `output_mem`	1	AGU write-strobe pipe (SRL16E)
Total, and steps 2+3+4+5 collapse to a constant 6		**8**	= `STORE_LATENCY`

So DRAIN's length is not a tuning parameter — it is **a mirror of the datapath's physical depth**. The AGU's write-strobe pipe is `STORE_LATENCY` deep and keyed off `compute_phase`, so for a 4-vector job the four `out_we` pulses land in the last four DRAIN cycles, the final one in the **very last** DRAIN cycle.

> **DRAIN HAS ZERO SLACK, BY CONSTRUCTION**
> Shorten it and `busy` drops early; the port-arbitration mux in `wrapper_full_system.sv` then hands the output SRAM back to the host while the AGU is still pulsing write strobes, and those writes are **SILENTLY DISCARDED**. Cut DRAIN by *k* cycles and you lose the last *k* results, with no error and no stall. The only legitimate way to shorten DRAIN is to make the datapath physically shallower.

11. The complete 17-cycle timeline

Everything above, assembled. Cycle 0 is the edge that samples `start`; N = 4.

```
cyc │ FSM phase │ what the AGU issues      │ what is inside the buffers / array
────┼───────────┼──────────────────────────┼──────────────────────────────────────────────
  0 │ IDLE      │ start sampled            │ array holds whatever it held before
  1 │ WLOAD     │ w_addr = 3               │ weight row 3 being read from weight_mem
  2 │ WLOAD     │ w_addr = 2               │ W[3] enters north edge, lands in row 0
  3 │ WLOAD     │ w_addr = 1               │ W[3]→row1, W[2]→row0
  4 │ WLOAD     │ w_addr = 0               │ W[3]→row2, W[2]→row1, W[1]→row0
  5 │ COMPUTE   │ act_addr = base+0        │ W[3]→row3 ... weights now ALL in place
    │           │                          │ (wload_q is still high this cycle - retimed by 1)
  6 │ COMPUTE   │ act_addr = base+1        │ A0 enters row 0.  Weight residue starts draining
  7 │ COMPUTE   │ act_addr = base+2        │ A0→row1/col1, A1→row0.   skew lines filling
  8 │ COMPUTE   │ act_addr = base+3        │ wavefronts A0,A1,A2 in flight
  9 │ DRAIN     │  -                       │ A3 entering; A0 reaches row 3 -> first C0 lane out
 10 │ DRAIN     │  -                       │ residue fully clear of the array
 11 │ DRAIN     │  -                       │ de-skew buffer holding the C0 staircase
 12 │ DRAIN     │  -                       │ last residue leaves the de-skew buffer
 13 │ DRAIN     │  -                       │ ◂ C0 aligned on result[] ; out_we #1 fires
 14 │ DRAIN     │  -                       │ ◂ C1 aligned            ; out_we #2
 15 │ DRAIN     │  -                       │ ◂ C2 aligned            ; out_we #3
 16 │ DRAIN     │  -                       │ ◂ C3 aligned            ; out_we #4  (last DRAIN cycle)
 17 │ DONE      │  -                       │ done pulses; busy already low
 18 │ IDLE      │  -                       │ host may read output_mem, or issue the next start
```

Reusing the weights removes rows 1–4 and shifts everything up by 4: COMPUTE at 1–4, DRAIN at 5–12, DONE at 13.

12. Where every cycle goes — the latency budget

Contribution	Cycles	Scales with	Can it be removed?
Weight load (WLOAD)	4	ROWS	**Yes — already is.** Skipped automatically when the weight matrix has not changed (§14).
Useful compute (COMPUTE)	N	N	No — this is the work.
of which: `a_mem` read	1	—	Only by going to asynchronous LUTRAM — costs a BRAM and hits the weakest timing path.
array traversal + de-skew	6	ROWS+COLS−2	**No.** This is the geometry of a 4×4 array. Removing the de-skew buffer does not help — the last column still emerges at +6.
write-strobe alignment	1	—	Coupled to the `a_mem` read cycle.
DRAIN (= the three rows above)	8	STORE_LATENCY	Only by making the datapath shallower. See the warning in §10.
DONE handshake	1	—	No — the pulse the host waits on.
Total	**N + 13**		**N + 9** reusing weights

13. Why the weight residue never corrupts a result

Earlier revisions of this design inserted a **FLUSH** phase of `ROWS + 2 = 6` cycles between WLOAD and COMPUTE. It has been removed, and the reasoning belongs in this document because it is really a dataflow argument.

The residue is real. During WLOAD every PE is doing `psum_q <= psum_in` (valid is low), while `psum_in` carries weight bytes shifting down the column. So when WLOAD ends, all sixteen `psum_q` registers contain weight bytes rather than zeros.

But it cannot reach a result, for two independent reasons:

Reason	Detail
1. Accumulation is fed from above, never from the local register	`adder_out = psum_in + product_ext` — `psum_q` is only ever an output. When the first real wavefront reaches a PE, that PE writes `psum_in + product`, discarding whatever was in `psum_q`. Stale content is overwritten, not accumulated into.
2. The residue and the results travel at the same speed, and the residue set off first	The residue drains out of the bottom at one row per cycle, then through the de-skew buffer, clearing the aligned `result[]` bus on cycle 12. The first `out_we` — the AGU's write-strobe pipe, keyed off `compute_phase` and `STORE_LATENCY` deep — does not fire until cycle 13. Two trains on the same track at the same speed, with the residue ahead: they cannot converge.

> **THE HONEST CAVEAT**
> Removing FLUSH removed the **CUSHION**, not a function. The separation between the last residue (cycle 12) and the first write strobe (cycle 13) is now exactly **ONE CYCLE**. Anyone who adds a pipeline stage anywhere in the `a_mem` → skew → array → de-skew → pack path must re-derive this table, because there is no longer any slack absorbing an off-by-one. The regression will catch it — `tb_accelerator_system_top` compares every lane of every result against an independently-computed golden `C = A × W` — but only if it is re-run.

14. Throughput, efficiency and the reuse path

The array does 16 MACs per cycle while COMPUTE is running — every PE busy, every cycle. At 100 MHz that is a peak of **1.6 GMAC/s (3.2 GOP/s)**. What you actually get depends on how much of the job is COMPUTE:

```
efficiency = N / (N + 13) loading weights
efficiency = N / (N + 9) reusing weights
```

N (vectors)	Cycles (load)	Cycles (reuse)	Efficiency load / reuse	Effective rate (load)
4 (one 4×4 matrix)	**17**	**13**	**23.5 % / 30.8 %**	0.38 GMAC/s
16	29	25	55.2 % / 64.0 %	0.88 GMAC/s
64	77	73	83.1 % / 87.7 %	1.33 GMAC/s
255 (max)	268	264	95.1 % / 96.6 %	1.52 GMAC/s

A single isolated 4×4 multiplication is the worst case this architecture has. The whole point of weight-stationary is amortising the load over many vectors; a 4-vector job barely amortises anything. That is the honest framing of the 23.5 % figure — it is not a defect, it is the demonstration being too small to show the architecture at its best.

How the reuse path works

`accelerator_system_top` keeps one bit, `weights_dirty`:

Event	Effect	Why
Reset	**set**	Every `weight_q` comes up zero, so the first job must load.
`host_w_we && !busy`	**set**	A host weight write that actually reached the SRAM. Host writes are masked while busy, hence the `!busy` gate — hammering the port mid-job cannot force a spurious reload.
Last WLOAD cycle	**cleared**	The array now matches `weight_mem`.

It drives `controller.load_weights`, so WLOAD is skipped whenever the array already holds the right matrix. The flag is deliberately **internal**: the module's port list is unchanged, so `fpga_top`, `vio_debug_top` and `accel_axi` all get the speed-up without modification and no host has to learn a new command bit. It is conservative in the safe direction — rewriting the same weight values still forces a reload.

15. Common misconceptions

Belief	Reality
"The skew buffer is a FIFO that buffers bursts."	No. It is a set of fixed-length shift registers with no flow control at all. It exists purely to make row *r*'s data arrive *r* cycles late. There is no back-pressure anywhere in this design.
"DRAIN flushes leftover garbage."	DRAIN waits for **real results** still in flight. The garbage it might also be pushing out is incidental — see §10 and §13.
"The weights must be reloaded for each job."	They survive indefinitely in `weight_q`. Only a host write to `weight_mem` makes a reload necessary, and the design detects that automatically.
"Deeper skew buffers would let it run faster."	The skew depths are fixed by the geometry (*r* in, *COLS−1−c* out). Changing them breaks alignment. What raises F_{max} is shortening the PE's combinational path — e.g. forcing the multipliers into DSP48E1 slices.
"The 128-bit output word means 128-bit datapaths."	It is four independent 32-bit lanes packed for storage only. Nothing in the array is wider than 32 bits.
"Efficiency is low, so the architecture is bad."	Efficiency is `N/(N+13)`. At N = 4 you are measuring the pipeline fill and drain, not the array. Stream 255 vectors and it is 95 %.

Sources. All structural claims are read from the RTL: `MAC.sv` (PE registers and the accumulate/bypass mux), `array.sv` (interconnect and boundary bindings), `input_buf.sv` / `output_skewbuf.sv` / `delay_pipe.sv` (skew depths), `controller.sv` (phase durations), `agu.sv` (address counters and the write-strobe pipe), `wrapper_array_buf.sv` / `wrapper_mem_arr_buf.sv` / `wrapper_full_system.sv` (latency constants, port arbitration, `weights_dirty`).
Measured. Cycle counts are observed, not assumed: `tb_controller` counts phase-enable clocks for `num_vectors` ∈ `{0,1,3,4,6,8,16}` on both the loading and skipping paths, and `tb_accelerator_system_top` T7/T8 measure `start` → `done` directly. Regression: **11/11 testbenches, 3 680 checks, 0 mismatches** (QuestaSim 2021.1).
Companions. `timing.pdf` §14 for performance and energy; `context_mkdwn/12_dataflow_and_timing.md` for the constant table; `architecture_and_dataflow.pdf` for the module-by-module reference.